

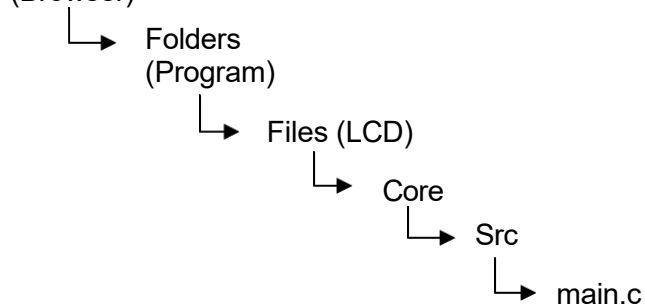
# Experiment No-07

**Aim:** Write an ARM Cortex-M0 assembly program to interface a seven-segment display and display it.

**Software Used:** STM32CubeIDE

## Steps to open STM32CubeIDE:

- 1) Launch
- 2) Micro-STM32F051
- 3) File → Import → General → Existing Workspace (Browser)



## Program:

### main.c

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file      : main.c
 * @brief     : Main program body
 * *****
 * @attention
 *
 * <h2><center>&copy; Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.</center></h2>
 *
 * This software component is licensed by ST under BSD 3-Clause license,
 * the "License"; You may not use this file except in compliance with the
 * License. You may obtain a copy of the License at:
 *
 * opensource.org/licenses/BSD-3-Clause
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
```

```

#include "main.h"
static void ssDisplay(unsigned int, unsigned char);
/* Private includes ----- */
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef ----- */
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ----- */
/* USER CODE BEGIN PD */
/* USER CODE END PD */

/* Private macro ----- */
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ----- */

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes ----- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code ----- */
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)

```

```

{
    int i,j,cnt,units,tens;

    /* USER CODE BEGIN 1 */
    unsigned char ssCode[]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
    HAL_Init();

    /* USER CODE BEGIN Init */

    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */

    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    /* USER CODE BEGIN 2 */

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        for(i=0;i<100;i++)
        {
            cnt = i;
            for(j=0;j<50;j++)
            {
                units = (cnt%10)&0x0F;
                ssDisplay(2,ssCode[units]);
                HAL_Delay(2);

                tens = cnt / 10;
            }
        }
    }
}

```

```

        tens = (tens%10)&0x0F;
        ssDisplay(1,ssCode[tens]);
        HAL_Delay(2);
    }
    HAL_Delay(10);
}

}
/* USER CODE END 3 */
}

ssDisplay(unsigned int disp,unsigned char val)
{
    //A
    if(val & 0x01)
        HAL_GPIO_WritePin(GPIOA, A_Pin,1);
    else
        HAL_GPIO_WritePin(GPIOA, A_Pin,0);

    //B
    if(val & 0x02)
        HAL_GPIO_WritePin(GPIOA, B_Pin,1);
    else
        HAL_GPIO_WritePin(GPIOA, B_Pin,0);

    //C
    if(val & 0x04)
        HAL_GPIO_WritePin(GPIOA, C_Pin,1);
    else
        HAL_GPIO_WritePin(GPIOA, C_Pin,0);

    //D
    if(val & 0x08)
        HAL_GPIO_WritePin(GPIOA, D_Pin,1);
    else
        HAL_GPIO_WritePin(GPIOA, D_Pin,0);

    //E
    if(val & 0x10)
        HAL_GPIO_WritePin(GPIOB, E_Pin,1);
    else
        HAL_GPIO_WritePin(GPIOB, E_Pin,0);

    //F

```

```

if(val & 0x20)
    HAL_GPIO_WritePin(GPIOB, F_Pin,1);
else
    HAL_GPIO_WritePin(GPIOB, F_Pin,0);

//G
if(val & 0x40)
    HAL_GPIO_WritePin(GPIOB, G_Pin,1);
else
    HAL_GPIO_WritePin(GPIOB, G_Pin,0);

//DP
if(val & 0x80)
    HAL_GPIO_WritePin(GPIOB, DP_Pin,1);
else
    HAL_GPIO_WritePin(GPIOB, DP_Pin,0);

//select display

//disp1
if(disp == 1)
    HAL_GPIO_WritePin(GPIOB, SL1_Pin,1);
else
    HAL_GPIO_WritePin(GPIOB, SL1_Pin,0);

//disp2
if(disp == 2)
    HAL_GPIO_WritePin(GPIOB, SL2_Pin,1);
else
    HAL_GPIO_WritePin(GPIOB, SL2_Pin,0);

}
/*=====*/

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

```

```

/** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */
RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSIState = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
{
    Error_Handler();
}
/** Initializes the CPU, AHB and APB buses clocks
 */
RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
        |RCC_CLOCKTYPE_PCLK1;
RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;

if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
{
    Error_Handler();
}
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOA, A_Pin|B_Pin|C_Pin|D_Pin, GPIO_PIN_RESET);

    /*Configure GPIO pin Output Level */
    HAL_GPIO_WritePin(GPIOB, E_Pin|F_Pin|G_Pin|DP_Pin

```

```

        |SL1_Pin|SL2_Pin, GPIO_PIN_RESET);

/*Configure GPIO pins : A_Pin B_Pin C_Pin D_Pin */
GPIO_InitStruct.Pin = A_Pin|B_Pin|C_Pin|D_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : E_Pin F_Pin G_Pin DP_Pin
                        SL1_Pin SL2_Pin */
GPIO_InitStruct.Pin = E_Pin|F_Pin|G_Pin|DP_Pin
                        |SL1_Pin|SL2_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

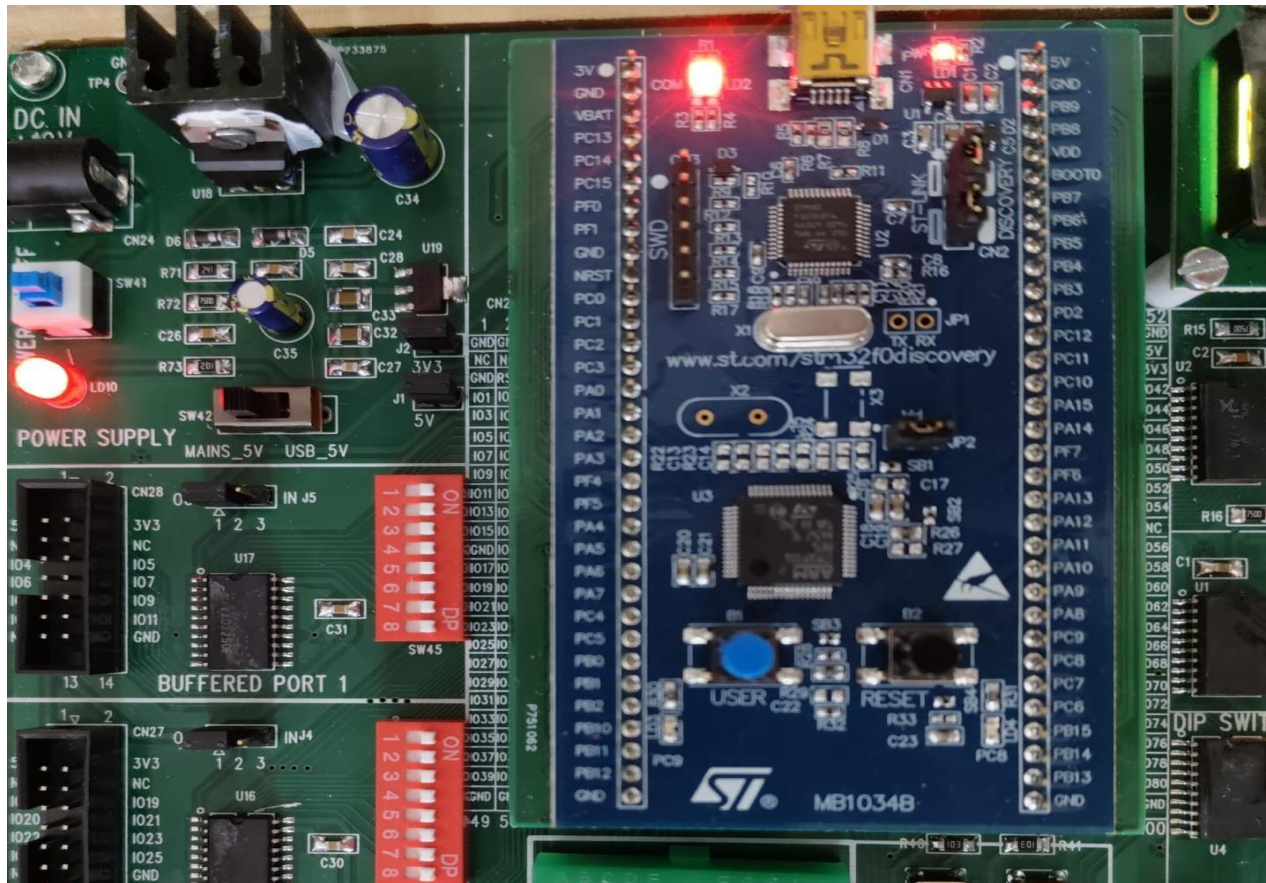
#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number

```

```
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line number,
       ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}

#endif /* USE_FULL_ASSERT */
```

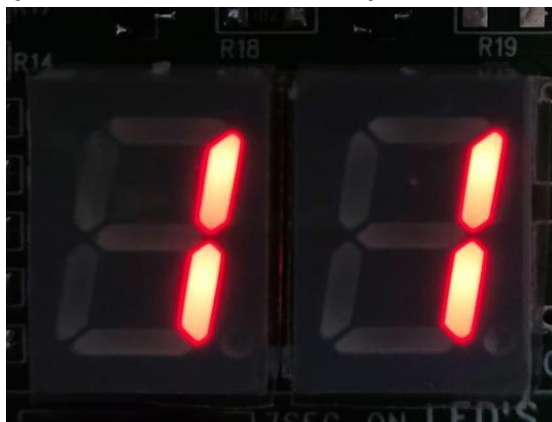
## Image of Microcontrollers:



## Results and Conclusion:

Output:

(COUNTER FROM 0-99)



Conclusion:

Thus, the ARM Cortex-M0 assembly program successfully interfaces the seven segment display and displays the required digit by sending the corresponding segment pattern through GPIO pins, demonstrating basic output interfacing.